

Capitolo 41 — PROT-012 — Verified Email Delivery & Idempotency

Pubblico	Lettori tecnici e non tecnici del WCM Process & Memory Book
Versione	FROZEN-41
Data master	2026-08-31
Stato	FROZEN
Source path	wcm/documentation/process-memory-book/ chapters/41_prot_012_verified_email_delivery_idempotency.md
Source SHA	6f9e473
Release	2026-09-04T09:45:54.479Z

GitHub main è la source of truth. Questo PDF è una release derivata: non costituisce approvazione né autorità.

PARTE VII — Il Libro dei Protocolli WCM Stato: FROZEN **Data:** 2026-08-31 **Scope:** WCM generale, domain-agnostic

41.0 Inviare non significa aver consegnato

Quando premiamo «Invia» su un messaggio, tendiamo naturalmente a considerare conclusa l'azione. Nella vita quotidiana questa semplificazione è spesso sufficiente. In un sistema operativo, però, può essere pericolosa.

Un comando di invio può partire e ricevere una risposta ambigua. Il provider può accettare il messaggio ma il tool può non restituire una conferma chiara. Un retry eseguito troppo presto può allora produrre un doppio invio. All'opposto, una risposta apparentemente positiva può non bastare a dimostrare che il messaggio corretto, con il destinatario corretto e gli allegati richiesti, sia davvero presente nella mailbox di invio.

PROT-012 — Verified Email Delivery & Idempotency esiste per separare tre fatti che non devono essere confusi:

```
CAPABILITY DISPONIBILE
≠
COMANDO DI INVIO ESEGUITO
≠
DELIVERY VERIFICATA
```

In parole semplici: avere la possibilità tecnica di mandare un'email non prova che l'email sia stata inviata; aver chiamato il comando di invio non prova che la consegna sia verificata.

Il protocollo introduce quindi una disciplina molto concreta: ogni messaggio logico deve poter essere riconosciuto in modo stabile, cercato prima dell'invio, verificato dopo l'invio e, in caso di incertezza, protetto da retry duplicati.

La regola da cui parte tutto è questa:

Un'email WCM è **DELIVERED soltanto quando esiste evidenza tecnica verificabile della sua presenza nella mailbox di invio prevista, con gli elementi obbligatori coerenti.**

41.1 Il problema che PROT-012 risolve

Immaginiamo di dover spedire una raccomandata. Consegniamo la busta allo sportello, ma la stampante della ricevuta si blocca. A quel punto non sappiamo ancora se la raccomandata sia stata registrata oppure no.

Le due reazioni impulsive sono entrambe sbagliate:

- dichiarare subito «invio fallito»;
- consegnare immediatamente una seconda busta identica.

La prima può produrre una falsa diagnosi. La seconda può produrre un duplicato.

Nel mondo digitale il problema è analogo, ma spesso meno visibile. Un connector o un provider può restituire un errore opaco dopo che il messaggio è già stato accettato. Oppure può restituire un identificatore tecnico che dimostra che qualcosa è avvenuto, ma non ancora tutto ciò che il workflow richiede.

Senza una disciplina esplicita, il sistema rischia di:

- dichiarare successi non provati;
- dichiarare fallimenti non provati;
- reinviare lo stesso messaggio più volte;
- dimenticare un allegato obbligatorio;
- confondere un problema di delivery con l'assenza della capability email;
- perdere continuità tra una run e la successiva.

PROT-012 trasforma quindi l'invio email da gesto istantaneo a **workflow verificabile**.

41.2 Perché esiste un delivery token

Per evitare i duplicati, bisogna poter riconoscere con certezza il messaggio logico che si sta cercando di consegnare.

Per questo il protocollo usa un **delivery_token**.

Il concetto è semplice: è un identificatore stabile associato non al singolo tentativo tecnico, ma al **messaggio logico**.

Se lo stesso messaggio viene tentato una seconda volta, il token resta lo stesso. Se invece il contenuto rappresenta un evento logico diverso, deve esistere un'identità distinta.

Il token deve essere:

- deterministico rispetto all'evento logico;
- riutilizzato negli eventuali retry dello stesso messaggio;
- sufficientemente specifico da non confondersi con altri messaggi;
- privo di segreti;
- ricercabile nel contenuto del messaggio.

Una riga del tipo:

```
Delivery-Token: <delivery_token>
```

permette al sistema di cercare quel messaggio nella mailbox di invio.

Il token non è una ricevuta di consegna al destinatario finale e non pretende di dimostrare che qualcuno abbia letto l'email. Serve a stabilire un fatto più preciso: **quel messaggio logico risulta effettivamente presente nel canale di invio previsto**.

41.3 Trigger: quando il protocollo si applica

PROT-012 si attiva quando un workflow WCM richiede una email operativa soggetta a verifica di delivery.

Il trigger non è «esiste una funzione email». Quello riguarda la capability tecnica ed è materia di PROT-011.

Il trigger di PROT-012 è invece:

```
EMAIL RICHIESTA
+
AZIONE AUTORIZZATA
+
DELIVERY DA ESEGUIRE O VERIFICARE
```

L'authority resta esterna al protocollo. PROT-012 non concede il diritto di comunicare, non decide chi debba ricevere un messaggio e non amplia lo scope del workflow.

Quando l'email è prevista e autorizzata, il protocollo governa **come stabilire in modo affidabile se la delivery è avvenuta**.

41.4 Gli input necessari

Prima di iniziare il flusso servono pochi elementi, ma devono essere chiari.

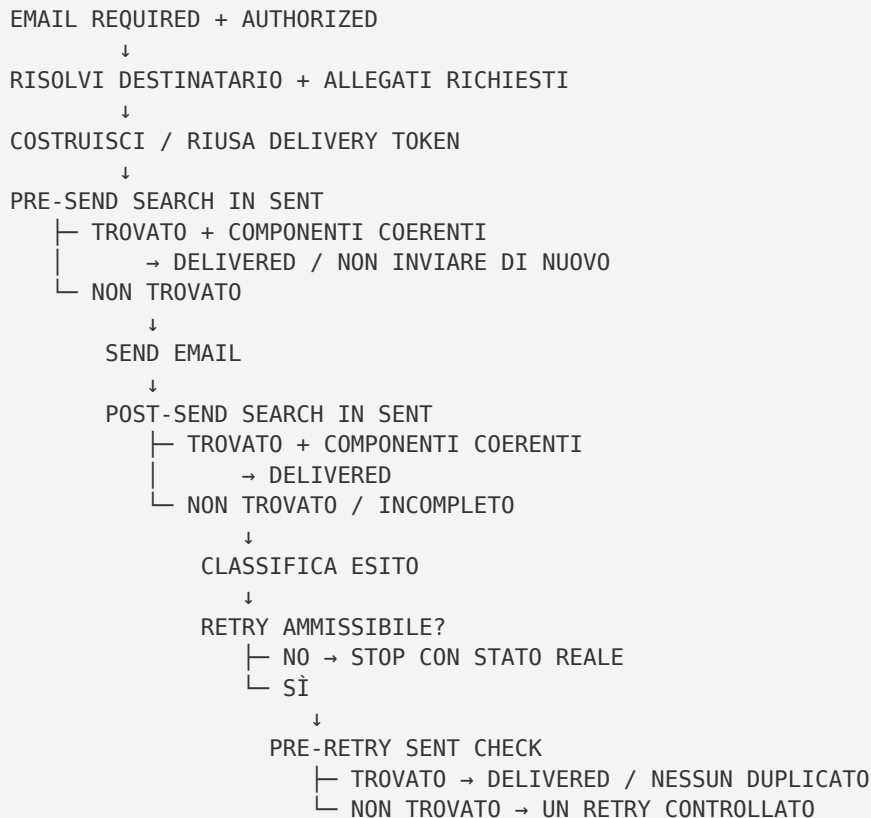
Gli input minimi sono:

- il messaggio logico da consegnare;
- il destinatario previsto dal workflow;
- l'eventuale modalità di risoluzione del destinatario;
- il subject coerente con l'evento;
- gli eventuali allegati obbligatori;
- il **delivery_token** stabile;
- l'authority applicabile;
- la capability email già verificata quando necessario.

Se mancano elementi obbligatori, il problema deve essere classificato prima dell'invio. Il protocollo non autorizza a sostituire arbitrariamente un destinatario, omettere un allegato richiesto o inventare dati mancanti per «far partire comunque» il messaggio.

41.5 Il flusso canonico

Il cuore di PROT-012 è una sequenza di verifiche che impedisce sia il falso successo sia il retry cieco.



La logica è volutamente prudente.

Prima di inviare si cerca il token. Questo protegge contro un messaggio già spedito in una run precedente o in un tentativo di cui non si possiede più una conferma affidabile.

Dopo l'invio si cerca di nuovo. Questo trasforma la mailbox di invio in una fonte di evidenza tecnica più durevole della sola risposta momentanea del comando.

41.6 Gate 1 — Il messaggio esiste già?

Il primo gate è il controllo pre-send.

Prima di eseguire un nuovo invio, il sistema cerca nella mailbox **Sent** — o equivalente del provider — il **delivery_token** atteso.

Se trova il messaggio e gli elementi obbligatori coincidono, il risultato è già:

```
DELIVERED
```

Non bisogna inviarlo di nuovo.

Questo gate è la prima difesa contro i duplicati. È particolarmente importante quando una run viene ripresa dopo un'interruzione o quando l'esito del tentativo precedente non era stato chiaramente registrato.

Il principio è semplice:

Prima di ripetere un'azione potenzialmente già riuscita, verifica se il suo effetto esiste già.

41.7 Gate 2 — La risposta del comando basta?

No, non sempre.

Il protocollo distingue tra un buon segnale tecnico e una prova sufficiente per il workflow.

Un message ID restituito dal tool è evidenza utile. Ma, quando il provider consente la verifica nella mailbox di invio, PROT-012 richiede il controllo successivo in **Sent**.

Questo perché il protocollo non vuole sapere soltanto se una chiamata tecnica ha restituito «success». Vuole verificare il risultato che interessa al workflow.

La domanda quindi non è:

«Il comando ha risposto bene?»

ma:

«Il messaggio logico atteso è presente dove dovrebbe risultare dopo l'invio?»

41.8 Gate 3 — Che cosa deve essere verificato in Sent

La presenza di una email qualsiasi non basta.

La verifica deve controllare almeno:

- **delivery_token** corretto;
- destinatario coerente;
- subject coerente con l'evento;
- presenza degli allegati obbligatori, quando previsti;
- filename degli allegati richiesti, quando rilevante.

In questo modo il sistema evita un falso positivo: trovare un messaggio con lo stesso argomento ma non quello previsto dal workflow.

La verifica non pretende di dimostrare tutto ciò che accade dopo l'uscita dal provider. Non dimostra lettura, comprensione o risposta del destinatario. Dimostra il livello di evidenza che il protocollo governa: **delivery verificabile nel canale di invio previsto**.

41.9 Gli allegati non sono un dettaglio

Quando un workflow richiede allegati, una email priva di uno di essi non rappresenta il completamento corretto del package.

Prima dell'invio occorre quindi verificare che gli artefatti richiesti:

1. esistano;
2. siano quelli corretti;

3. siano effettivamente previsti;
4. vengano allegati senza aggiunte arbitrarie.

Dopo l'invio, la verifica in **Sent** deve controllare i filename attesi quando la capability lo consente.

Se il messaggio esiste ma manca un allegato obbligatorio, il protocollo vieta di classificare come completata la delivery di quel package.

È un principio generale: **consegnare una parte non equivale a consegnare il risultato richiesto**.

41.10 Le quattro classificazioni di delivery

PROT-012 evita il semplice schema successo/fallimento perché, nei sistemi reali, esistono stati intermedi importanti.

DELIVERED

Il messaggio è presente in **Sent** con token, destinatario e componenti obbligatori coerenti.
Azione: registrare il successo e non reinviare lo stesso messaggio logico.

DELIVERY_UNVERIFIED

L'esito non è dimostrabile con evidenza sufficiente.

Per esempio: il comando ha restituito una risposta ambigua e la verifica in **Sent** non può essere completata oppure non produce dati sufficienti.

Azione: non dichiarare né successo né failure definitiva. Conservare token e next verification/recovery action quando il workflow richiede persistenza.

TEMPORARY_DELIVERY_BLOCK

La capability esiste, ma un ostacolo concreto e potenzialmente transitorio impedisce la delivery: autenticazione scaduta, rate limit, outage o altro errore tecnico contingente verificato.

Azione: quando la delivery è necessaria alla stop condition, il workflow resta riprendibile. Questo stato non deve essere trasformato in **CAPABILITY_GAP**.

DELIVERY_FAILED

Esiste evidenza di un fallimento non meramente transitorio del messaggio logico: per esempio un destinatario verificato come non valido o un componente obbligatorio che non può essere prodotto.

Azione: stop o escalation secondo il workflow applicabile. Il protocollo non autorizza retry infiniti.

41.11 Gate 4 — Quando è ammesso un retry

Il retry non è il comportamento predefinito.

PROT-012 ammette **al massimo un retry controllato nello stesso ciclo**, e solo se sono vere tutte queste condizioni:

1. il messaggio non risulta già in **Sent** con lo stesso token;
2. l'azione resta autorizzata;
3. il failure mode è concretamente retryable;
4. input e allegati sono ancora validi.

Immediatamente prima del retry la ricerca del token in **Sent** deve essere ripetuta.

Questa seconda ricerca può sembrare ridondante, ma protegge proprio dal caso più insidioso: il primo invio è partito davvero, mentre la risposta del tool è rimasta ambigua.

Senza questo controllo, un retry «prudenziale» potrebbe essere la causa stessa del doppio invio.

41.12 Errori opachi: descrivere solo ciò che sappiamo

Un sistema affidabile deve saper convivere con l'incertezza senza trasformarla in una diagnosi inventata.

Fraasi generiche come «email non disponibile», «invio bloccato» o «provider non consentito» non devono diventare conclusioni definitive se l'evidenza tecnica non le supporta.

Quando la capability esiste ma l'esito non è dimostrabile, la classificazione corretta è:

```
DELIVERY_UNVERIFIED
```

Questa disciplina può sembrare più cauta, ma in realtà rende il sistema più preciso. Dire «non so ancora se è stata consegnata» è organizzativamente migliore di inventare un successo o un fallimento.

41.13 Relazione con PROT-011 — Capability Evidence Check Before Block

PROT-011 e PROT-012 operano su due problemi diversi.

PROT-011 risponde alla domanda:

| La capability email esiste ed è disponibile?

PROT-012 risponde invece:

| Dato che la capability esiste, il messaggio logico è stato consegnato in modo verificabile e senza duplicati?

La relazione è quindi:

EMAIL CAPABILITY PRESUNTA ASSENTE
→ PROT-011

EMAIL CAPABILITY PRESENTE
MA DELIVERY DA ESEGUIRE / VERIFICARE
→ PROT-012

Una delivery failure non dimostra l'assenza della capability.

Questa distinzione evita che un errore locale venga promosso a limite strutturale dell'organizzazione.

41.14 Relazione con PROT-004 — Idempotenza

PROT-012 applica al dominio email un principio già presente nel WCM: la stessa intenzione logica non deve produrre effetti duplicati solo perché il sistema viene ripetuto o ripreso.

Il **delivery_token** svolge il ruolo di identità stabile del messaggio logico. La ricerca pre-send e pre-retry verifica se l'effetto esiste già.

La relazione concettuale può essere espressa così:

```
IDENTITÀ LOGICA STABILE
+
CHECK DELL'EFFETTO GIÀ ESISTENTE
+
RETRY CONTROLLATO
=
DELIVERY IDEMPOTENTE ENTRO IL CONTRATTO DEL PROTOCOLLO
```

Il protocollo non promette idempotenza universale di qualunque sistema email. Definisce una disciplina WCM per ridurre in modo verificabile il rischio di invii duplicati nel perimetro governato.

41.15 Relazione con PROT-009 — Workflow contiguo

Quando una email è un output obbligatorio di un workflow, la sua delivery deve essere trattata senza confondere lo stato del lavoro con lo stato della comunicazione.

Il workflow può aver raggiunto correttamente un risultato operativo, mentre la delivery può essere ancora **DELIVERY_UNVERIFIED** o temporaneamente bloccata.

PROT-009 richiede continuità fino alla vera stop condition. PROT-012 fornisce la classificazione tecnica necessaria per capire se l'obbligo di comunicazione sia realmente chiuso oppure debba restare riprendibile.

In altre parole:

```
STATO DEL LAVORO
≠
STATO DELLA DELIVERY
```


Il reporting deve mantenere separati i due piani.

41.16 Output ed evidence minima

Quando la delivery è materialmente rilevante, il risultato deve poter essere ricostruito.

L'evidence minima prevista dal protocollo comprende, quando applicabile:

```
delivery_token
recipient resolution mode
send attempt outcome
Sent verification outcome
attachment verification outcome
final delivery state
```

Non devono essere esposti segreti, token di autenticazione o identificatori tecnici non necessari.

L'obiettivo è rendere auditabile la conclusione, non accumulare dati sensibili.

41.17 Failure mode principali

Falso successo

Il sistema dichiara **DELIVERED** solo perché il comando di invio ha risposto positivamente.

Conseguenza: chi legge lo stato crede conclusa una delivery non verificata.

Retry cieco

Una risposta ambigua viene interpretata come failure e il messaggio viene reinviato senza controllo in **Sent**.

Conseguenza: doppio invio.

Token instabile

Ogni retry genera una nuova identità per lo stesso messaggio logico.

Conseguenza: la deduplicazione perde significato.

Allegato mancante classificato come successo

L'email è presente in **Sent**, ma il package obbligatorio è incompleto.

Conseguenza: il workflow registra un risultato più forte di quello realmente ottenuto.

Delivery failure trasformata in capability gap

Un problema di invio viene descritto come assenza della capability email.

Conseguenza: una difficoltà locale diventa falsamente un limite strutturale.

Diagnosi inventata

Un errore opaco viene tradotto in una spiegazione non supportata dall'evidenza.

Conseguenza: la Persistent Organizational Memory conserva una causa falsa.

41.18 Maturity e limiti

La baseline canonica di PROT-012 è qualificata come:

VALIDATED BY GOVERNANCE
FIELD VALIDATION IN PROGRESS

Questo significa che il protocollo è parte della governance corrente, ma la validazione sul campo non deve essere presentata come universalmente conclusa.

I criteri di field validation indicati dalla fonte canonica includono scenari come:

- email semplice con ritrovamento in **Sent**;
- email con allegati e verifica dei filename richiesti;
- stesso delivery token senza doppio invio;
- errore ambiguo con **Sent** check prima di qualsiasi retry;
- errore temporaneo non classificato come **CAPABILITY_GAP**;
- reporting che non dichiara successo senza evidenza tecnica.

Il protocollo dipende inoltre dalle capacità concrete del provider o connector. Se una verifica prevista non è tecnicamente disponibile, il sistema non deve fingere di averla eseguita: deve rappresentare correttamente il livello di evidenza realmente raggiunto.

PROT-012 non dimostra lettura del messaggio da parte del destinatario, non sostituisce le regole di authority e non definisce il contenuto semantico delle comunicazioni. Governa affidabilità, verificabilità e idempotenza della delivery email nel perimetro WCM.

41.19 Source map

Il capitolo deriva dalla baseline canonica corrente:

- **wcm/process-book/protocols/PROT-012_VERIFIED_EMAIL_DELIVERY_IDEMPOTENCY.md** — fonte tecnica primaria;
- **wcm/process-book/protocols/PROT-011_CAPABILITY_EVIDENCE_CHECK_BEFORE_BLOCK.md** — relazione capability vs delivery;
- **wcm/process-book/protocols/PROT-009_CONTIGUOUS_WORKFLOW_EXECUTION.md** — continuità e stop condition;
- **wcm/process-book/protocols/PROT-004_CANONICAL_DISPATCH_IDEMPOTENCY.md** — principio trasversale di identità/idempotenza;
- **WCM_AGENT_START.md** — source precedence, authority e routing generale.

Il capitolo non introduce nuovi stati, nuovi gate o nuove authority rispetto alla baseline tecnica: traduce il protocollo in forma editoriale e pedagogica.

41.20 La regola da ricordare

Se di questo capitolo rimanesse una sola idea, dovrebbe essere questa:

Non confondere il tentativo di invio con la consegna verificata: identifica il messaggio, cerca prima, invia, verifica dopo e non ripetere mai alla cieca.

PROT-012 rende esplicita una qualità essenziale di un sistema affidabile: non basta compiere un'azione; bisogna poter dimostrare quale effetto abbia realmente prodotto.